MLOps Assignment 2: LLMOps — API Cost, Latency, and Prompt Version Management

Student: Kush Gunendrasing Munot

BITS ID: 2025ae05915

Course: MLOps

Date: 2026-11-04

Github Repo Link - LINK

1. Objective and setup

This experiment measures the latency, token usage, cost and prompt-version behaviour of the EduAssist AI assistant. Five fixed academic questions were sent to three prompt versions using the same model and settings. The prompts were versioned in the MLflow Prompt Registry and the best one was promoted with the alias `production`.

The model was `gemini-3.1-flash-lite` (snapshot reported by the API: `gemini-3.1-flash-lite`), called through the Google Gemini API v1beta native `generateContent` endpoint. Standard pricing was $0.25 per 1M input tokens and $1.50 per 1M output tokens (USD, checked 2026-11-03). Temperature was 0 and the output cap was 1,024 tokens. MLflow 3.17.2 ran in Podman; the notebook used Python 3.12.

2. Latency (Task 1)

Five baseline requests used prompt v1. Mean latency was 9.21 s, median 6.72 s, minimum 2.97 s, maximum 20.40 s and sample standard deviation 7.00 s. The slowest request was Q4 at 20.40 s; it also produced the most output (729 tokens), and latency correlated strongly with output length ($r = 0.93$, $n = 5$).

Output length does not fully explain it. The same prompts produced identical token counts in the later v1 evaluation, yet Q4 took only 2.91 s and the mean was 4.12 s. Most of Q4's delay was therefore transient. Network jitter and provider queueing are possible reasons this experiment cannot separate.

3. Tokens and cost (Task 2)

No thinking tokens were reported. The average cost per request was $0.001008, which projects to $10.08 for 10,000 requests. Output tokens made up 99.5% of the cost, because the output price is six times the input price.

The recommendation is to shape and cap response length. Reducing the average output from 668 to about 250 tokens would lower the average cost to about $0.00038 per request, or $3.80 per 10,000 requests. The trade-off is lost detail and possible JSON truncation, so `finishReason` and `valid_json` must be monitored.

4. Prompt evaluation (Task 3)

The experiment `eduassist_llmops_assignment` evaluated prompt `eduassist-academic-qa` versions 1–3 on the same five questions. v2 added the academic-assistant role, concise wording and undergraduate-level language. v3 required only valid JSON with `answer`, `confidence` and `key_points`.

Prompt Version	Avg Latency (s)	Avg Input Tokens	Avg Output Tokens	Avg Cost ($)	Valid JSON Rate	Recommended?
v1	4.12	20.2	668.4	0.001008	0.0	No
v2	3.43	44.2	465.4	0.000709	0.0	No
v3	2.57	85.2	176.0	0.000285	1.0	Yes

`valid_json` is the share of responses that parse with `json.loads` and match the schema, with no repair of code fences. The same validator was used for all versions, so 0.0 for v1 and v2 is expected.

5. Promotion decision

I promoted **v3 (registry version 3)** to the `production` alias. It is the only version that meets the JSON contract the downstream application needs: 5 of 5 responses were valid, against 0 of 5 for v1 and v2. Prose output would require a fragile repair step.

v3 uses more input tokens (85.2 versus 20.2 and 44.2) because it embeds the schema, but far fewer output tokens (176.0 versus 668.4 and 465.4). Since output costs six times as much, its average cost is $0.000285 per request, against $0.001008 and $0.000709, or about $2.85 versus $10.08 and $7.09 per 10,000 requests. Its average latency was also lowest at 2.57 s, although latency was noisy.

On quality, my manual review of all 15 responses scored v3 at 4.0 out of 5, against 4.4 for v1 and 4.8 for v2. All answers were correct, but v3 is shorter and less detailed. I accept that trade-off because validity is a hard requirement; the alias can be moved back if monitoring shows problems.

6. Limitations

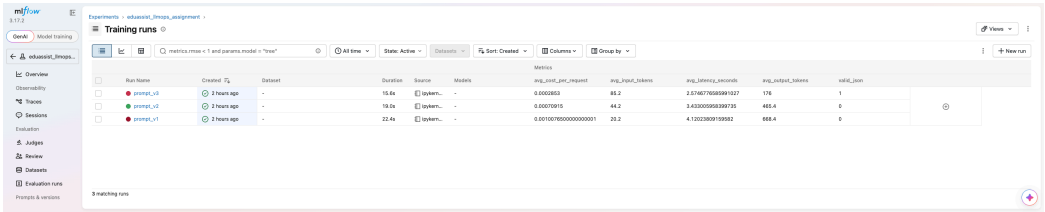
Five questions is a small sample, so statistics are indicative. Output is non-deterministic and latency varied from 2.5 s to 20 s for the same prompts. Prices exclude tax. `valid_json` checks structure, not correctness, so the manual review remains necessary.

7. Evidence screenshots

The screenshots below are from the MLflow UI at http://127.0.0.1:5001 after the final notebook run. The same files are in the `screenshots/` folder.

7.1 Experiment runs and metrics

experiment eduassist_llmops_assignment with runs prompt_v1, prompt_v2, prompt_v3 and the five required metrics

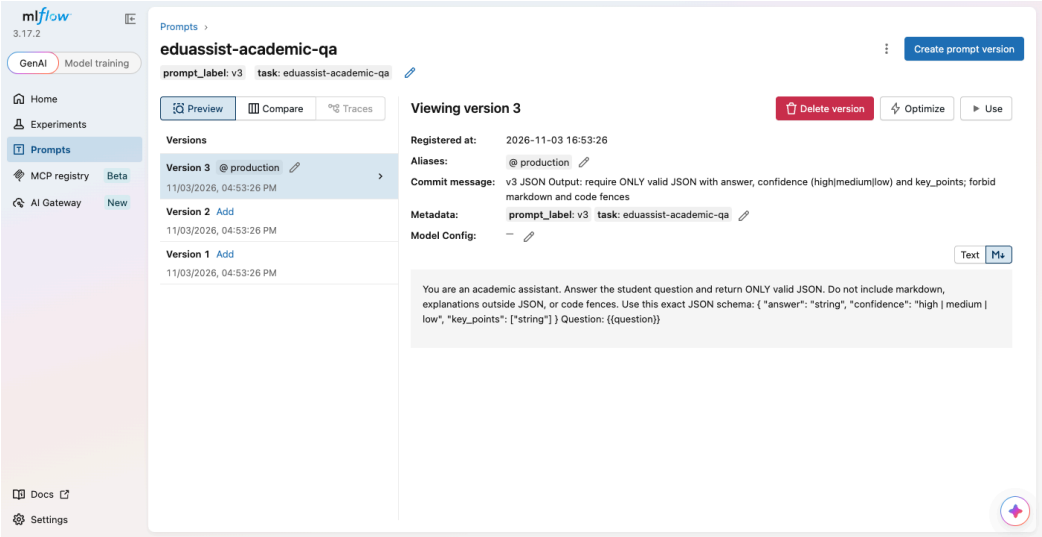


experiment eduassist_llmops_assignment with runs prompt_v1, prompt_v2, prompt_v3 and the five required metrics

Screenshot 01: experiment eduassist_llmops_assignment with runs prompt_v1, prompt_v2, prompt_v3 and the five required metrics

7.2 Prompt Registry versions

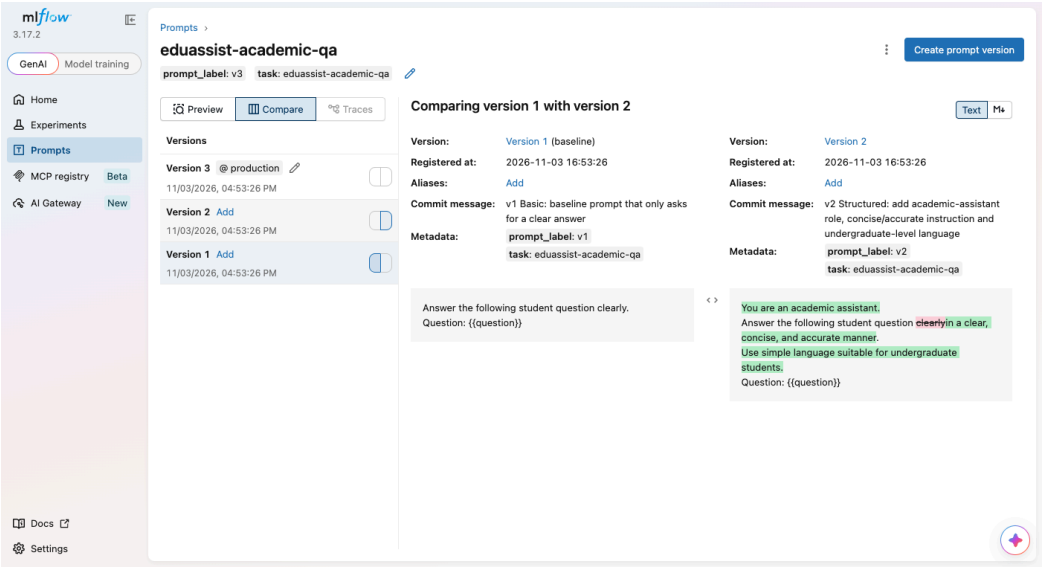
prompt eduassist-academic-qa with versions 1, 2 and 3



Screenshot 02: prompt eduassist-academic-qa with versions 1, 2 and 3

7.3 Prompt diff v1 to v2

comparison of version 1 and version 2

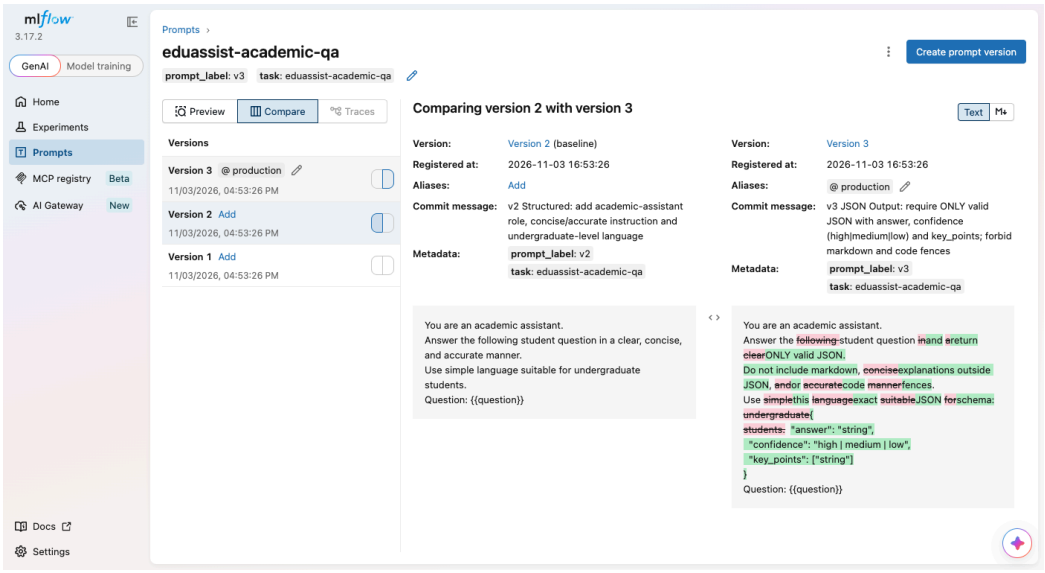


comparison of version 1 and version 2

Screenshot 03: comparison of version 1 and version 2

7.4 Prompt diff v2 to v3

comparison of version 2 and version 3

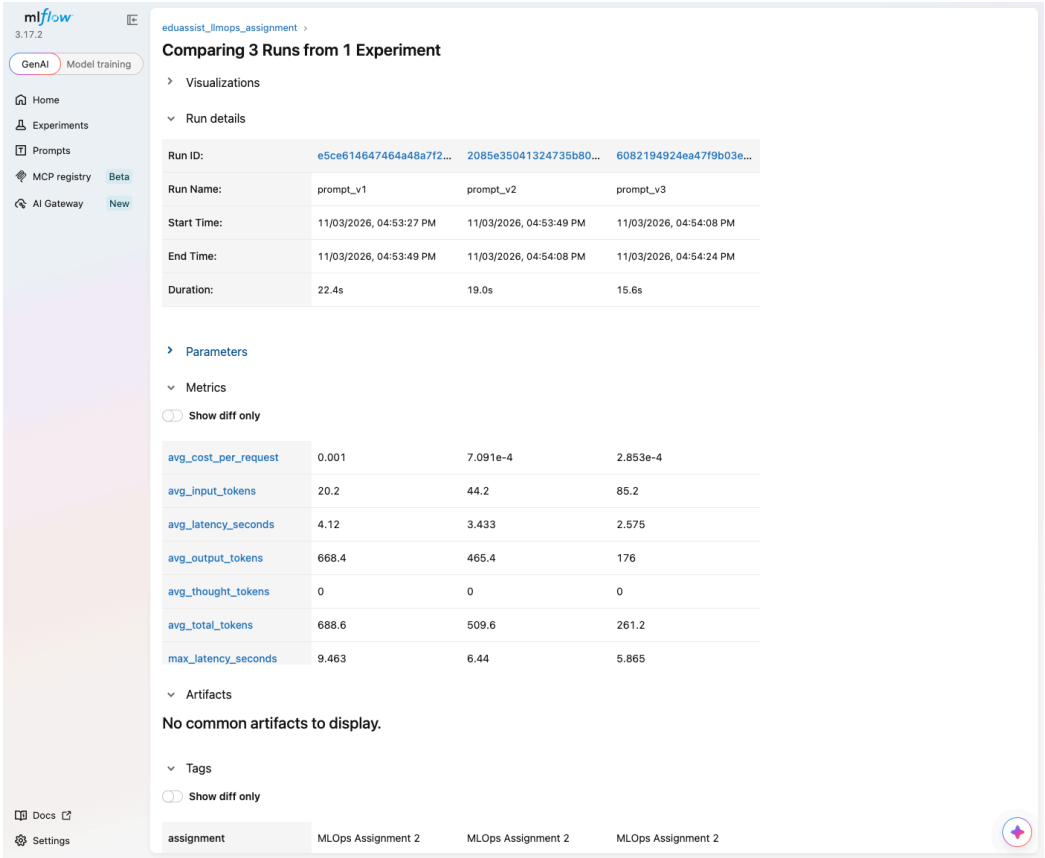


comparison of version 2 and version 3

Screenshot 03b: comparison of version 2 and version 3

7.5 Run metrics comparison

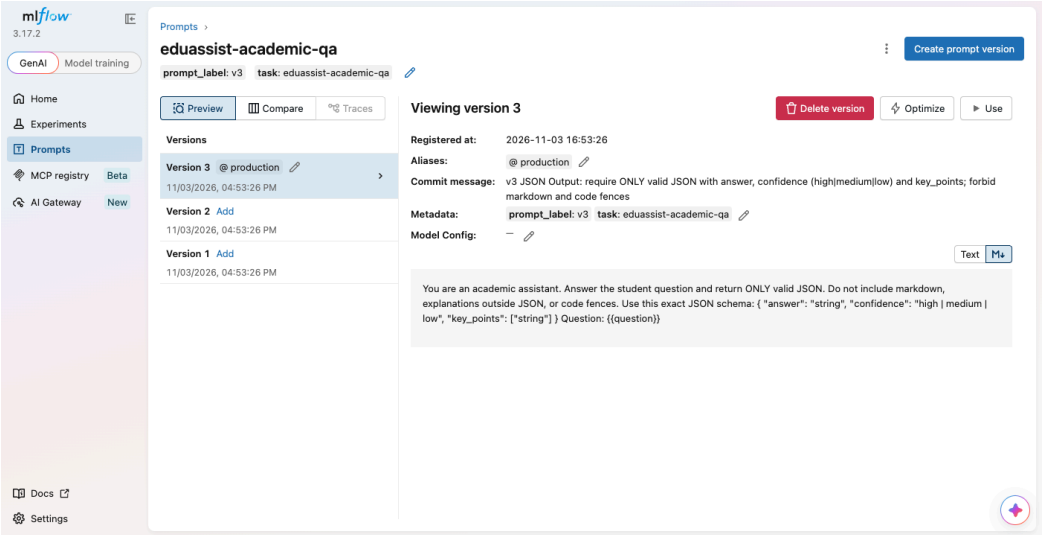
side-by-side metrics for the three runs



side-by-side metrics for the three runs

Screenshot 04: side-by-side metrics for the three runs

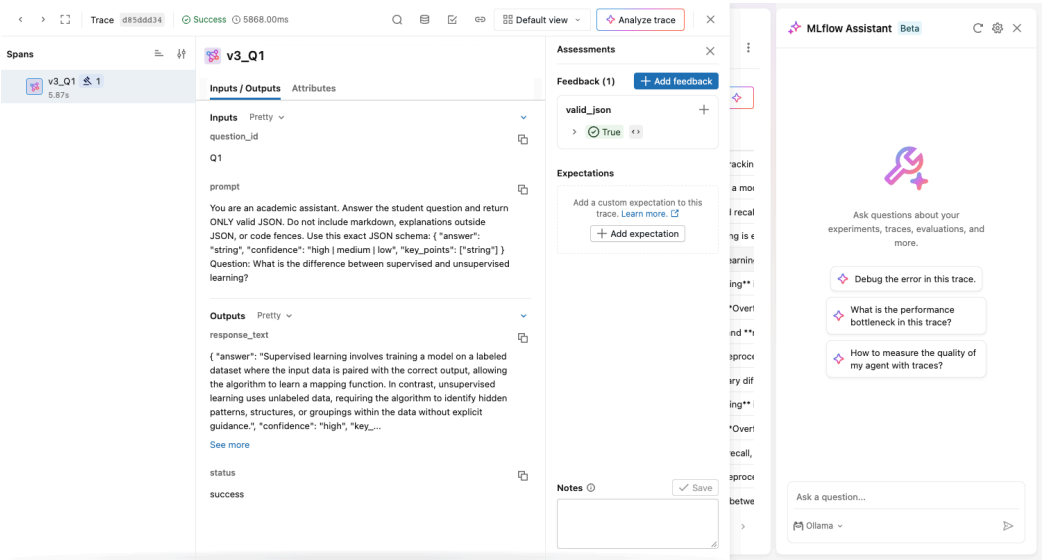
7.6 Production alias



Screenshot 05: alias production attached to prompt version 3

7.7 Trace with valid_json feedback

v3 trace showing valid_json = True



v3 trace showing valid_json = True

Screenshot 06: v3 trace showing valid_json = True